CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA SOUZA
FACULDADE DE TECNOLOGIA DA BAIXADA SANTISTA "RUBENS LARA"
CURSO SUPERIOR DE TECNOLOGIA EM CIÊNCIA DE DADOS
PROGRAMA DE INICIAÇÃO CIENTÍFICA

PETERSON CARARA JUNIOR

FORMALIZAÇÃO DE MÁQUINAS DE ESTADOS FINITOS COMPONÍVEIS EM HASKELL: SEGURANÇA DE TIPOS E COMPLETUDE FUNCIONAL VIA MONADS E TIPOS ALGÉBRICOS

Santos - SP
2025

PETERSON CARARA JUNIOR

FORMALIZAÇÃO DE MÁQUINAS DE ESTADOS FINITOS COMPONÍVEIS EM HASKELL: SEGURANÇA DE TIPOS E COMPLETUDE FUNCIONAL VIA MONADS E TIPOS ALGÉBRICOS

Projeto de pesquisa apresentado à Faculdade de Tecnologia da Baixada Santista “Rubens Lara”, do Curso Superior de Tecnologia em Ciência de Dados, no Programa de Iniciação Científica.

Orientador: Prof. Felipe Canarozzo Lourenço

Santos - SP
2025

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ADT	Algebraic Data Type (Tipo de Dados Algébrico)
CREM	Crème de la Crem: Composable Representable Executable Machines
DDD	Domain-Driven Design
DFSM	Deterministic Finite State Machine
FATEC BS	Faculdade de Tecnologia da Baixada Santista Rubens Lara
FSM	Finite State Machine (Máquina de Estados Finitos)
I/O	Input/Output
PFP	Programação Funcional Pura

Sumário

LISTA DE ABREVIATURAS E SIGLAS 1

1 INTRODUÇÃO 3
1.1 PROBLEMA DE PESQUISA 4
1.2 JUSTIFICATIVA DO TEMA 4
1.2.1 Hipóteses 4
1.3 OBJETIVOS 5
1.3.1 Objetivo Geral 5
1.3.2 Objetivos Específicos 5

2 REFERENCIAL TEÓRICO INICIAL 6
2.1 Máquinas de Estados Finitos e a Questão da Mutabilidade 6
2.2 Haskell e a Garantia de Corretude em Tempo de Compilação 6
2.2.1 Tipos de Dados Algébricos (ADTs) 7
2.2.2 Pattern Matching e Completude Funcional 7
2.3 Teoria das Categorias: Monads, Estado Puro e a Categoria Kleisli 7
2.3.1 Monads: Origem Categorial, Leis e a Abstração do Efeito de Estado . . 7
2.3.2 A Categoria Kleisli para a Composição de Transições 8
2.4 Composabilidade Arquitetural e o Framework CREM 8

3 PROCEDIMENTOS METODOLÓGICOS 9

4 CRONOGRAMA 10

5 REFERÊNCIAS 11

1 INTRODUÇÃO

As Máquinas de Estados Finitos (FSMs) constituem uma das abstrações mais duradouras e essenciais da Ciência da Computação. Elas fornecem o formalismo matemático rigoroso para a modelagem de sistemas cujo comportamento é estritamente determinado por um conjunto discreto de estados e regras de transição. O modelo é crucial para a especificação do comportamento de sistemas reativos, abrangendo desde protocolos de rede até a lógica de domínio em arquiteturas de *software* complexas, permitindo, inclusive, **"construir arquiteturas de software como uma composição de máquinas de estados"** (Perone, p. 1).

No entanto, a tradução desse modelo abstrato para linguagens de programação de propósito geral do paradigma imperativo, como C++, é frequentemente marcada pela dependência de variáveis mutáveis, tipicamente campos de classe ou variáveis globais, para armazenar o estado atual. Essa prática compromete a pureza do modelo matemático e resulta em uma implementação que, segundo Malakhovski e Shalyto (p. 1), **"não consegue aplicar diretamente os métodos utilizados na implementação de máquinas de estados finitos em linguagens de programação imperativas"** em um ambiente funcional. A dependência da mutabilidade e de tipos fracos para o estado expõe o sistema a erros lógicos e transições inválidas, delegando a responsabilidade de verificar a corretude à fase de testes em tempo de execução.

O presente projeto de pesquisa propõe resolver essa discrepância fundamental através da **formalização e implementação de FSMs na linguagem de programação funcional pura Haskell**. Haskell, sendo uma linguagem **"puramente funcional"** (Hutton, p. 2), exige que a função de transição seja implementada como uma função pura. O diferencial reside no seu sistema de tipos avançado: a utilização de **Tipos de Dados Algébricos (ADTs)**, essenciais para a modelagem de dados (Milewski, p. 21), permite que os conjuntos finitos de estados e eventos sejam mapeados diretamente para tipos de dados. Essa correspondência de tipo não é apenas uma convenção, mas uma imposição que viabiliza a **"verificação da correção do código das funções de transição no estágio de compilação pelo próprio compilador"** (Malakhovski & Shalyto, p. 20) por meio da verificação de exaustividade do *pattern matching*.

Além da segurança de tipos básica, a abordagem em Haskell permite a exploração de conceitos avançados para o tratamento do estado. A capacidade de sequenciar transições e lidar com o estado de forma pura é resolvida através de **Monads**, uma abstração que **"expressa concisamente certos programas que manipulam estado"** (Wadler, p. 1) e que permite a **"mudança de estado de forma pura"** através de estruturas como a `State Monad` (Milewski). Esse arcabouço monádico, estruturado pela **Categoria Kleisli** (Maia, p. 15), fornece o formalismo necessário para a **composição** segura de transições.

A capacidade de **composabilidade** é o ponto culminante desta abordagem. O *framework* **Crem** (Crème de la Crem: Composable Representable Executable Machines), desenvolvido por Marco Perone, ilustra como os *advanced type-level features* de Haskell podem ser usados para **"especificar transições de estado permitidas e proibidas"** (Perone, p. 1). Essa especificação

transforma as regras de domínio em propriedades intrínsecas ao tipo de dados, permitindo a **"composição de máquinas de estados"** independentes (Perone), resultando em um sistema robusto e modular, onde a lógica de domínio é validada pelo compilador antes da execução.

1.1 PROBLEMA DE PESQUISA

Considerando o descompasso entre a formalidade matemática das FSMs e a fragilidade de tipos nas implementações tradicionais, o problema de pesquisa é formulado como:

> De que forma a linguagem de programação funcional pura Haskell, explorando seu sistema de tipos avançado (Tipos de Dados Algébricos) e suas abstrações de ordem superior (Monads e Categoria Kleisli), pode ser utilizada para obter uma formalização de Máquinas de Estados Finitos que ofereça garantias verificáveis de correção (*type safety*) e completude (*completeness*) na especificação das funções de transição, e que suporte a composição de FSMs complexas de maneira modular e representável?

1.2 JUSTIFICATIVA DO TEMA

A relevância do projeto reside em transferir a verificação da corretude da FSM da fase de testes para a fase de compilação. Malakhovski e Shalyto (p. 20) afirmam que uma das vantagens da implementação funcional é a **"verificação da correção do código das funções de transição no estágio de compilação pelo próprio compilador, graças ao uso de tipos de dados algébricos para representação de eventos, estados"**.

Além disso, a estrutura de Haskell, baseada na Teoria das Categorias (Milewski, p. 2), permite lidar com o estado de forma pura através de **Monads**, que foram originalmente desenvolvidas por teóricos de categorias para **"expressar concisamente certos aspectos da álgebra universal"** (Wadler, p. 1). Monads, formalmente um **"Monóide na Categoria dos Endofuntores"** (Jahn, p. 3), permitem a **"mudança de estado de forma pura"** (Milewski) e possibilitam a **composição** segura de transições via a Categoria Kleisli (Maia; Jahn). Essa composabilidade é crucial, permitindo **"construir arquiteturas de software como uma composição de máquinas de estados"** independentes, o que contribui para a modularidade e a coerência do domínio do *software*.

1.2.1 Hipóteses

O sistema de tipos de Haskell oferece uma **Segurança de Tipos** superior, onde o uso dos Tipos de Dados Algébricos (ADTs) e o *pattern matching* conseguem impor, em tempo de compilação, a **proibição de estados e transições logicamente impossíveis**. Essa garantia se estende à **Completude Funcional**, onde o *pattern matching* exaustivo assegura que a função de transição trata todas as combinações válidas de (Estado $\times$ Evento), o que é uma superioridade na verificação da lógica da FSM em relação às linguagens imperativas (Malakhovski &

Shalyto). Finalmente, postula-se a **Hipótese de Composabilidade e Modulação**: a aplicação de abstrações categoriais como **Monads** e recursos avançados de *type-level* de Haskell (como os utilizados em Crem) permite a criação de **Máquinas de Estados Finitos Componíveis** de forma puramente funcional, resultando em uma arquitetura modular cujas regras de domínio são validadas pelo compilador (Perone).

1.3 OBJETIVOS

1.3.1 Objetivo Geral

O objetivo geral desta Iniciação Científica é formalizar e implementar Máquinas de Estados Finitos Determinísticas (DFSMs) em Haskell, explorando o sistema de tipagem avançado e as estruturas funcionais da linguagem para garantir a segurança e a completude na definição das funções de transição, com ênfase na demonstração de um modelo que suporte a composição de sistemas.

1.3.2 Objetivos Específicos

Para alcançar o objetivo geral de formalizar e implementar FSMs de forma segura e componível em Haskell, o projeto se desdobra nos seguintes objetivos específicos.

O primeiro objetivo visa o alicerce do projeto: **definir modelos de dados em Haskell** que traduzam com precisão os elementos formais de uma FSM. Esta formalização será realizada através da criação de **Tipos de Dados Algébricos (ADTs)**, nos quais os conjuntos finitos de Estados (Q) e Eventos (Σ) serão mapeados diretamente para Tipos Soma. Essa técnica visa incorporar o rigor do modelo matemático ao código, seguindo o princípio de que ADTs são **"o conceito mais importante"** na modelagem funcional (Milewski, p. 21).

O segundo objetivo trata da validação da lógica interna da máquina: **implementar o mecanismo de processamento de sequências de entrada e a função de transição (δ)**. Serão exploradas as funções de alta ordem e, sobretudo, o ***pattern matching*** exaustivo para definir a função δ : Estado $\times$ Evento $\rightarrow$ Estado. Essa abordagem é fundamental para atingir a **Hipótese de Completude Funcional**, pois o compilador de Haskell garantirá a exaustividade, assegurando que todo par válido $(\text{Estado}, \text{Evento})$ seja tratado, transferindo a **"verificação da correção do código das funções de transição para o estágio de compilação"** (Malakhovski & Shalyto, p. 20).

Em seguida, o terceiro objetivo foca na gestão do estado e na composabilidade: **explorar a aplicação do *State Monad* e da Categoria Kleisli**. Será demonstrado como a `State Monad` permite gerenciar a sequência de mudanças de estado de forma pura, encapsulando o estado e garantindo a **"mudança de estado de forma pura"** (Milewski). Esta etapa é crítica para lidar com o histórico de transições. Além disso, a **Categoria Kleisli** será utilizada como base teórica para **compor transições monádicas**, fornecendo o formalismo para encadear logicamente as transições de estado, como sugerido por Milewski e Maia (p. 15).

Finalmente, o quarto objetivo é a validação prática da abordagem: **demonstrar a composabilidade de máquinas de estados por meio de um estudo de caso**. Este estudo de caso ilustrará como a tipagem de Haskell pode ser usada para garantir transições válidas e o encapsulamento do domínio, permitindo que a lógica do *software* seja estruturada como uma **"composição de máquinas de estados"** modulares (Perone, p. 1). O resultado esperado é um sistema onde as regras de transição — inclusive a **"especificação de transições de estado permitidas e proibidas"** (Perone, p. 1) — são validadas pelo compilador, confirmando o potencial das estruturas funcionais para o desenvolvimento de *software* robusto.

2 REFERENCIAL TEÓRICO INICIAL

O presente referencial teórico estabelece a base de conhecimento necessária para a formalização das Máquinas de Estados Finitos (FSMs) no ambiente funcional puro. A abordagem é construída sobre a fusão de quatro domínios conceituais: a Teoria de Autômatos, a Programação Funcional Pura (PFP), a Teoria das Categorias e as Arquiteturas de *Software* Componíveis.

2.1 Máquinas de Estados Finitos e a Questão da Mutabilidade

As FSMs são o modelo canónico para descrever sistemas sequenciais e reativos, cuja dinâmica é regida por uma **quíntupla formal** $(Q, \Sigma, \delta, q_0, F)$, onde Q é o conjunto finito de estados, Σ é o alfabeto de entrada (eventos), δ é a rigorosa função de transição, q_0 é o estado inicial, e F é o conjunto de estados finais. A função $\delta : Q \times \Sigma \to Q$ é o coração do sistema, definindo como o autômato se move entre os estados.

A implementação tradicional desse modelo em linguagens de programação imperativas, como C++ ou Java, cria uma divergência perigosa entre o modelo e o código. A representação do estado recorre, invariavelmente, a variáveis mutáveis — tipicamente **"campo de classe (se a programação orientada a objetos for usada) ou uma variável global"** (Malakhovski & Shalyto, p. 1). Além disso, os identificadores de estado são frequentemente representados por abstrações de baixo nível, como *strings* ou inteiros, que carecem de garantias de tipo. Essa dependência de estado mutável e a fragilidade de tipos resultam numa tradução que não consegue aplicar **"diretamente os métodos utilizados na implementação de máquinas de estados finitos em linguagens de programação imperativas"** (Malakhovski & Shalyto, p. 1) no rigoroso domínio funcional, expondo a FSM a estados logicamente impossíveis ou transições inválidas que só são detectadas em tempo de execução, através de pesados testes unitários.

2.2 Haskell e a Garantia de Corretude em Tempo de Compilação

O projeto propõe a **Programação Funcional Pura (PFP)** como a solução para este descompasso. **Haskell**, uma linguagem puramente funcional, baseia-se na imutabilidade e na ausência de efeitos colaterais. Sua filosofia exige que a função de transição (δ) seja implementada como

uma **função pura**, uma transformação que recebe o estado e o evento e retorna, deterministicamente, o novo estado, preservando a imutabilidade do sistema e produzindo *software* **"claro, conciso e correto"** (Hutton, p. 2).

2.2.1 Tipos de Dados Algébricos (ADTs)

A segurança formal em Haskell é fornecida por seu sistema de tipos avançado, ancorado nos **Tipos de Dados Algébricos (ADTs)**. Os ADTs permitem que os conjuntos finitos Q e Σ da FSM sejam mapeados diretamente para **Tipos Soma**, que Milewski (p. 21) define como o **"conceito mais importante"** para a modelagem em Haskell. Por exemplo, se uma FSM possui estados *Ligado* e *Desligado*, o Tipo Soma **data Estado = Ligado | Desligado** traduz essa finitude para o sistema de tipos. Essa codificação evita o uso de **"strings e inteiros"** (Malakhovski & Shalyto, p. 1) como identificadores de estado, garantindo que o compilador imponha a integridade estrutural da máquina.

2.2.2 Pattern Matching e Completude Funcional

A função de transição é implementada por meio de ***pattern matching***, uma funcionalidade poderosa para examinar a estrutura dos dados (Hutton, p. 49). O aspecto crucial é que o compilador de Haskell executa uma **verificação de exaustividade** neste *pattern matching*. Essa checagem garante que **toda combinação possível** de $(\text{Estado} \times \text{Evento})$ tenha sido tratada explicitamente na definição da função δ. Caso uma combinação válida seja omitida, o compilador alerta o programador com um erro, o que se traduz na **"verificação da correção do código das funções de transição no estágio de compilação"** (Malakhovski & Shalyto, p. 20), confirmando a **completude (*completeness*)** da FSM antes da execução.

2.3 Teoria das Categorias: Monads, Estado Puro e a Categoria Kleisli

Em um ambiente funcional puro, onde as transições de estado devem ser implementadas como funções puras, o desafio inerente é a gestão da **sequência de transições** ao longo do tempo. O projeto aborda este desafio recorrendo à **Teoria das Categorias**, que é a disciplina matemática que fornece os fundamentos lógicos para a linguagem Haskell. O princípio orientador é que a **"Composition is the Essence of Programming"** (Milewski, p. 8), e a Teoria das Categorias define o arcabouço para a composição segura e coerente.

2.3.1 Monads: Origem Categorial, Leis e a Abstração do Efeito de Estado

As **Monads** são o dispositivo arquitetónico que permite a Programação Funcional Pura gerir efeitos colaterais, como o estado, exceções ou I/O, sem violar a imutabilidade. Conforme Philip Wadler (p. 1) explica, Monads proporcionam **"as indulgências de impureza com as bênçãos de pureza"** ao encapsular o "efeito".

A sua origem é estritamente matemática: Monads foram inventadas na década de 1960 por teóricos de categorias para **"expressar concisamente certos aspectos da álgebra universal"**. A definição categorial formal e mais abstrata estabelece uma Monad como um **"Monóide na Categoria dos Endofuntores"** (Jahn, p. 3). Esta definição, embora abstrata, é fundamental, pois garante que a estrutura Monádica satisfaça as **Leis das Monads** (identidade e associatividade), assegurando que o encadeamento de efeitos seja previsível e deterministicamente seguro.

Para a FSM, a **`State Monad`** é a Monad específica empregada. Ela resolve o dilema do estado ao modelar a transição como uma função de transformação de estado, na qual a função de transição pura é encapsulada. Isto permite que a **"mudança de estado de forma pura"** (Milewski) seja realizada, pois a função não *modifica* o estado, mas sim *calcula* o novo estado a partir do antigo, retornando o par (Valor, Novo Estado) e mantendo a imutabilidade fora da abstração monádica.

2.3.2 A Categoria Kleisli para a Composição de Transições

Embora Monads resolvam o problema do estado encapsulado, a composição dessas funções monádicas (as transições de estado) não pode ser realizada pela composição de funções padrão. É aqui que entra a **Categoria Kleisli**.

A Kleisli é uma construção categorial específica que é projetada para compor **morfismos monádicos**, ou seja, funções "sujas" que carregam o efeito do *Monad*. Em vez de funções A→B, a Kleisli compõe morfismos do tipo A→MB (Maia, p. 15), onde M é o Functor Monádico (a `State Monad` no nosso caso) que carrega o estado. Essa categoria fornece as regras e, mais importante, o operador de composição monádica (>=>) que automatiza a logística da sequenciação.

O uso da Categoria Kleisli é crucial para o projeto, pois ele fornece o formalismo para **encadear múltiplas transições de estado da FSM** de forma segura e elegante. A Monad, sob a estrutura da Kleisli, garante que o estado do autômato seja automaticamente passado, transformado e extraído em cada etapa da sequência de eventos. Isso não só simplifica a sintaxe, como garante, a partir dos fundamentos da Teoria das Categorias, que a composição de transições monádicas seja sempre **coerente** e **associativa**, um pré-requisito fundamental para a **composabilidade** de FSMs complexas em arquiteturas modulares.

2.4 Composabilidade Arquitetural e o Framework CREM

A validação final da abordagem reside na sua capacidade de suportar arquiteturas de *software* complexas. O projeto alinha-se aos princípios do *Domain-Driven Design* (DDD), utilizando a FSM como um agregado de domínio verificável.

O *framework* **Crème de la Crem (CREM)** é um exemplo prático que demonstra o potencial máximo da PFP nesse contexto. O CREM utiliza os **"recursos avançados de *type-level* de Haskell"** para permitir que o *software* seja estruturado como uma **"composição de máquinas**

de estados" independentes (Perone, p. 1). Essa modularidade permite que subsistemas de domínio sejam desenvolvidos de forma isolada e testada, e combinados de forma segura.

A maior contribuição do CREM é a capacidade de **codificar regras de negócio como tipos**. Ele permite ao usuário **"especificar transições de estado permitidas e proibidas"** (Perone, p. 1) diretamente na assinatura de tipos. Isso significa que as restrições mais complexas da lógica de domínio da FSM são transformadas em **restrições de tipo**, e qualquer violação de uma regra de negócio resultará em um erro de compilação, e não em um erro de *runtime*. Isso garante que a **"implementação de um sistema se mantenha sincronizada com a sua representação gráfica"** (Perone, p. 1), estabelecendo um nível de **representabilidade** e integridade arquitetural superior, onde a lógica de negócio é verificada pelo compilador.

3 PROCEDIMENTOS METODOLÓGICOS

A metodologia deste projeto será de natureza mista, combinando o rigor da pesquisa formal com a validação prática, pautando-se numa **abordagem qualitativa** para analisar a fundo o fenômeno da formalização de Máquinas de Estados Finitos (FSMs) no contexto da programação funcional pura. O trabalho se desenvolverá em duas vertentes principais: a **Pesquisa Bibliográfica** e a **Pesquisa Aplicada (ou Tecnológica)**. A Pesquisa Bibliográfica fornecerá a base formal, coletando e analisando o referencial teórico sobre FSMs, Programação Funcional Pura (Haskell), Tipos de Dados Algébricos (ADTs), Teoria das Categorias (Monads, Categoria Kleisli) e os princípios do *framework* CREM. Esta base teórica é crucial para a construção do modelo computacional.

A **Pesquisa Aplicada** será a fase de experimentação, focada no desenvolvimento de um artefato de *software* (código em Haskell) para comprovar as hipóteses de segurança e composabilidade. Os procedimentos serão organizados em quatro fases interligadas que visam, em última instância, cumprir todos os objetivos específicos do projeto.

A **Fase 1: Revisão Teórica e Formalização Inicial** visa estabelecer o alicerce do código. Inicialmente, será concluída a revisão aprofundada dos fundamentos da FSM e de como a imutabilidade da PFP se traduz numa alternativa superior ao modelo imperativo. Em seguida, o primeiro objetivo específico será atendido pela **modelagem de ADTs**, onde os conjuntos de Estados (Q) e Eventos (Σ) de uma FSM canónica serão declarados como Tipos Soma em Haskell. O segundo objetivo será alcançado com a **implementação da função de transição pura** δ utilizando ***pattern matching*** exaustivo, o que é um procedimento metodológico chave para validar a **Hipótese de Completude Funcional**, demonstrando que o compilador exige o tratamento de todas as combinações válidas $(\text{Estado} \times \text{Evento})$.

A **Fase 2: Gestão do Estado e Sequenciação Monádica** endereçará o desafio da pureza na manipulação de estado, cumprindo o terceiro objetivo específico. O estudo da `State Monad` será traduzido na implementação, garantindo que a sequência de transições seja realizada através de uma função que calcula o novo estado a partir do antigo, mantendo a **"mudança de**

estado de forma pura" (Milewski). Esta fase incluirá a sequenciação de eventos utilizando a notação *do-notation* de Haskell, demonstrando como a **Categoria Kleisli** fornece o formalismo necessário para a composição segura dessas transições monádicas.

A **Fase 3: Demonstração da Composabilidade e Segurança** é a fase de validação prática da arquitetura. Será selecionado um **Estudo de Caso de Domínio** que exija a composição de múltiplas FSMs independentes. O procedimento metodológico aqui será aplicar os conceitos da Categoria Kleisli para **compor as máquinas**, provando a modularidade arquitetural. O ponto de maior rigor será o **Teste de Invalidez de Tipagem**, que envolverá a tentativa intencional de codificar uma transição proibida pela lógica do domínio da FSM. A documentação do erro de compilação resultante comprovará a **Hipótese de Segurança de Tipos**, validando que o sistema de tipos de Haskell é capaz de **"especificar transições de estado permitidas e proibidas"** (Perone, p. 1) em tempo de compilação.

Por fim, a **Fase 4: Análise e Conclusão** consistirá na comparação qualitativa dos resultados do modelo funcional puro implementado com o modelo imperativo tradicional, focando na **Segurança de Tipos**, **Completude** e **Modularidade** alcançadas. O código-fonte desenvolvido servirá como a evidência principal para a elaboração final do relatório, que sistematizará o conhecimento adquirido e apresentará a validação conclusiva das hipóteses de pesquisa.

4 CRONOGRAMA

O cronograma a seguir descreve o planejamento de todas as etapas da pesquisa ao longo do período de desenvolvimento do projeto de Iniciação Científica.

Etapas	**fev**	**mar**	**abr**	**mai**	**jun**	**jul**	**ago**	**set**	**out**	**nov**	**dez**
Revisão bibliográfica aprofundada	X	X									
Modelagem de ADTs e função de transição		X	X								
Implementação de State Monad e Kleisli			X	X							
Estudo de caso e composição de FSMs				X	X						
Testes de invalidez de tipagem					X	X					
Análise comparativa e documentação						X	X				
Elaboração do relatório final							X	X	X		
Entrega e apresentação										X	X

5 REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS - ABNT. **NBR 6023**: Informação e documentação - Referências - Elaboração. Rio de Janeiro, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS - ABNT. **NBR 10520**: Informação e documentação - Citações em documentos - Apresentação. Rio de Janeiro, 2023.

BOCCHINO JR., R. L. **Monads, Categories, and Computation**. [S.l.]: [s.n.], 2025.

HUTTON, G. **Programming in Haskell**. 2. ed. Cambridge: Cambridge University Press, 2016.

JAHN, M. **Monads and their Applications in Haskell**. 2020. Trabalho de Conclusão de Curso (Bachelor's Thesis) – Department of Mathematics, Technische Universität München, Munich, 2020.

MAIA, D. **Applied Category Theory: Monads and Haskell**. 2022.

MALAKHOVSKI, Y. M.; SHALYTO, A. A. [Máquinas de estados finitos em linguagens de programação funcional pura]. RSDN Magazine, São Petersburgo, n. 1, 2009.

MILEWSKI, B. **Category Theory for Programmers**. [S.l.]: [s.n.], 2020. E-book.

OLIVEIRA, A. G. de. **Haskell: Uma introdução à programação funcional**. São Paulo: Casa do Código, 2017.

PERONE, M. Crème de la Crem: Composable Representable Executable Machines. 2023.

WADLER, P. Comprehending Monads. *Mathematical Structures in Computer Science*, v. 2, n. 4, p. 461-493, 1992.